HIPAAChecker: Automating Compliance in Mobile Health Applications

Ryan Ramirez

COSC 6998: Professional Project

Abstract

Mobile and web-based health applications are capable of managing a vast amount of electronic protected health information (ePHI), yet a majority of them are deployed without first being validated against the technical safeguards mandated by the Health Insurance Portability and Accountability Act (HIPAA). Formal compliance audits are quite expensive, making it difficult for non-enterprise level developers to align with HIPAA compliance. To address this challenge, this project analyzes and validates the findings of the platform HIPAAChecker, a static analysis tool that is designed to automate compliance verification. The methodology involves evaluating HIPAA-specific rule mappings, evaluating compliance violations across a curated dataset of applications to validate the HIPAAChecker framework. This project, and its results demonstrates HIPAAChecker's viability as an accessible, developer-centric solution that reduces the burden of security validation and promotes greater trust across the mHealth market.

Keywords

HIPAA, mHealth, Compliance, ePHI, CVSS, Static Analysis

1. Introduction

1.1 Problem Background

In recent years, the increasing popularity of mobile and web-based mHealth applications has changed how healthcare services are accessed and provided. These applications often handle sensitive patient data, including diagnoses, prescriptions, and biometric records. Such information is regarded as electronic protected health information (ePHI), defined under HIPAA. Despite the sensitive nature of this data, many mHealth applications are deployed without proper validation against HIPAA's technical safeguards. Such vulnerabilities can expose patient data and subject both developers and healthcare professionals to regulatory and reputational risks.

Typically, formal compliance audits rely on manual processes that are often time-consuming, highly expensive, and resource-intensive. This can prove challenging for developers of small to mid-sized teams when attempting to incorporate security compliance into their development. While a few enterprise-level HIPAA compliance auditors exist on the market today, they are frequently built for large-scale hospital systems, making them difficult to adopt for teams of small or independent teams.

Thus, developers are left without practical tools to verify whether their applications are truly HIPAA compliant.
This lack of resources creates a vulnerability gap between regulation requirements, and the tools available to meet the demand. Furthermore, it increases the widening divide between public trust and secure data handling.
As the number of health-related applications continues to increase, so too does the need for accessible, automated compliance verification tools.

1.2 Purpose and Scope

The purpose of this project is to assess the reliability and applicability of HIPAAChecker as a tool for automated HIPAA compliance testing. The project explores whether a platform like HIPAAChecker can provide meaningful feedback on whether an application satisfies HIPAA's technical safeguards. The platform's ability to evaluate data protection practices, authorization mechanisms, and activity monitoring is key to this evaluation.
This assessment focuses on validating HIPAAChecker's effectiveness across its full range of supported platforms. This includes mobile Android applications using the following frameworks: PHP (Laravel), Python (Django), Java (Spring Boot), Ruby on Rails, Express.js, and .NET.
For the purpose of this project, a subset of a dataset of over 200 applications was evaluated. Each application was of a unique framework. In doing so, it highlights how well it identifies common safeguard violations and whether its outputs can guide developers toward more secure design choices. The HIPAAChecker platform is examined for its practical value in real-world development.
In particular, how well HIPAAChecker aligns with HIPAA's technical rule categories, how its scanning and scoring mechanisms function in practice, and how accessible the tool is for developers with limited security experience. Additionally, focus is given to the clarity of its output, its potential integration into developer workflows, and the degree to which the tool can provide actionable insights during routine testing.
This work builds on prior academic research and applies the tool across a small, curated sample of testing applications. The aim of HIPAAChecker is to contribute to the effort of reducing the barrier-to-entry in ensuring HIPAA compliance and encourage secure health software policies.

2. Literature Analysis

2.1 Android Safeguard Studies

Prior research was conducted on a study of 200 Android mHealth applications to determine their adherence to HIPAA's technical safeguard requirements. The study applied static analysis techniques, using decompilation, rule mapping, and data evaluation to measure how real application behavior aligned with twelve security controls. Each application was examined for implementation patterns related to access control, encryption, user authentication,and secure transmission of ePHI.
The findings revealed widespread non-compliance of a majority of the applications[1].
Many failed to implement basic access control mechanisms, relied on outdated or weak encryption algorithms, or neglected secure transmission practices. Even in more popular health applications, the study observed insecure network communications and limited enforcement of role-based access. These violations occurred in combination, compounding risk to patient data integrity and confidentiality [1].

Despite using an earlier version of HIPAAChecker, this study reveals a clear disconnection between available security frameworks and their use in development. This suggests that without practical tools integrated into the development process, compliance measures are likely to be overlooked.

2.2 Laravel-Based mHealth Apps

Another study conducted a static analysis of 200 open-source Laravel-based mHealth applications to evaluate their alignment with HIPAA's technical safeguard requirements. The study focused on web applications hosted on public Github repositories and were labeled as providing healthcare or wellness related functionality. The objective was to examine how well these web applications aligned with HIPAA's technical safeguard requirements through static analysis.

The findings from [1] were expanded upon, and the results indicated that 86.7 percent of the applications tested failed to meet the minimum compliance expectations. In fact, many of the applications again lacked technical safeguards around user authorization, data confidentiality, and activity monitoring. It was found that projects often omitted encryption for sensitive records, or failed to enforce secure communication standards such as HTTPS. Even with Laravel's native security features, many applications failed to implement them effectively.

The study attributed the compliance vulnerabilities to the way in which developer teams engage with security features. For example, built-in safeguards are often underused or bypassed due to lack of clarity around HIPAA expectations, or a misalignment between development and compliance needs. This suggests that effective enforcement of technical safeguards just as much on developer guidance as it does on platform capability.

2.3 Existing Compliance Tools Analysis

Currently, many enterprise-level platforms exist that offer compliance automation of frameworks like HIPAA, SOC2, GDPR, etc. For example, IBM Guardium and Secureframe are two such platforms that provide extensive audit logging, policy enforcement, and real-time monitoring capabilities. The problem however, is that these platforms are for large-scale infrastructure like hospitals or enterprise environments. Despite their capabilities, such tools are often very inaccessible for early-stage or small development teams. Integrations of these platforms can take months, and licensing costs can typically exceed what smaller teams are capable of. Additionally, these platforms are not designed to provide compliance feedback within the development stage and are instead intended for compliance and data governance at the system level, rather than static or dynamic analyses within development.

The lack of accessible tools is particularly concerning, given the continued growth of digital health and mobile health applications. It was reported that over 300,000 health-related apps are now listed across Apple's App Store and Google Play. The majority of these apps are not built by large healthcare institutions [5].

As smaller teams continue to deploy health-related applications, the need for accessible and lightweight compliance tools that deliver HIPAA-specific insight becomes all the more urgent. Tools like HIPAAChecker address this by focusing on

technical safeguard detection within the source code itself.

3. System Description

3.1 User Workflow

Figure 1. User Workflow Diagram

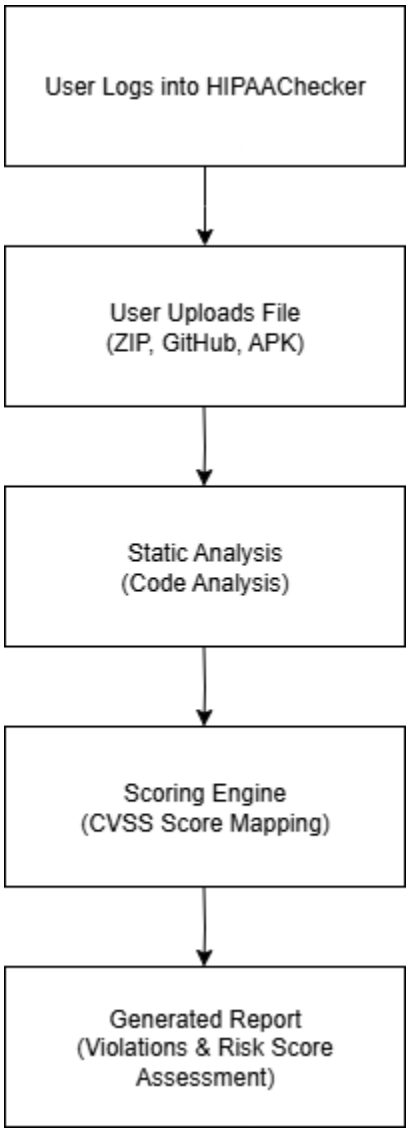

Figure 1 illustrates the HIPAAChecker user workflow.

HIPAAChecker is designed to support developers in performing compliance assessments early on in development. It offers a browser-based interface through which users can upload project files or a Github link for analysis. Alternatively, entire compressed project directories can be scanned, allowing developers to test pre-built applications or an entire source code repository.
Once uploaded, the system performs a multi-stage scan that includes static code inspection, rule-matching against HIPAA technical safeguards, and evaluation of common implementation patterns. All of this is done without any installation or configuration required, which greatly reduces the barrier to entry for teams unfamiliar with security testing tools or protocols.

Afterwhich, the scan results are presented as a scored report with detailed feedback on individual safeguards. Categories such as access control, data transmission security, and logging practices are evaluated, thereby allowing developers to precisely identify specific areas of concern that triggered the compliance flag.
The output is provided in non-technical language to more easily assist developers in identifying compliance vulnerabilities early, even if they lack knowledge of compliance protocols.

3.2 Android App Features

The Android version of HIPAAChecker mirrors the browser-based version in functionality and ease of usability. With the Android app, users can select and scan installed mHealth applications directly from their device, allowing for a real-time evaluation of HIPAA compliance directly on-device, without ever needing access to the original source code.
Once an application is selected, the tool automatically extracts the android

application file and analyzes its decompiled data.
This allows HIPAAChecker to evaluate flaws in implementation patterns that may trigger compliance flags or warnings.
The results are then mapped to the twelve technical safeguard categories as defined by HIPAAChecker. Since the Android version only works with apps that have been installed on-device, it allows developers to quickly assess the degree to which deployed mHealth applications remain compliant.

3.3 Twelve Technical Safeguards

HIPAA's technical safeguard requirements serve as the foundation for evaluating whether mHealth applications protect patient data in accordance with HIPAA. While HIPAA formally defines five technical safeguard categories, HIPAAChecker expands these into twelve more refined implementation checks for its analysis. In this way, it allows the specific compliance trigger to be highlighted, such as improper PHI encryption or automatic logoff procedures for example, which are not explicitly separated in the original HIPAA language. Each detected issue is then mapped back to one of these twelve categories in the final report.

The adjacent table presents the five parent categories defined by HIPAA, along with the twelve safeguard checks mapped by HIPAAChecker under each parent category.

HIPAA Defined Category	HIPAAChecker Safeguard
1. Access Control	**Authorization:** Limits ePHI access based on user role **Unique User Identification:** Assigns a unique ID to each user to track activity **Emergency Access Procedure:** Defines methods for accessing ePHI in urgent scenarios **Automatic Logoff:** Logs users out after a period of inactivity
2. Audit Controls	**Audit Logging:** Records and examines system activity involving ePHI
3. Integrity Controls	**Data Integrity:** Ensures that ePHI is not altered without authorization **Unauthorized Alteration/Destruction:** Verifying ePHI has not been improperly destroyed
4. Authentication	**User/Entity Authentication:** Verifies the identity of those attempting to access ePHI
5. Transmission Security	**Network Security:** Secures ePHI during transmission **PHI Encryption:** Protects ePHI using cryptographic methods. **Decryption:** Ensures secure access to encrypted ePHI **Transport Layer Encryption:** Applies encryption to the data transmission layer

Table 1: HIPAA Categories and HIPAAChecker Safeguards

4. Methodology

4.1 Static Analysis Method & Common Vulnerability Scoring System (CVSS)

During a scan, HIPAAChecker applies static analysis to evaluate whether an application aligns with HIPAA's technical safeguard requirements.
For Laravel web-based applications, the tool inspects source code and configuration files, routing logic, middleware layers, and database interactions. These components are then analyzed for patterns that may trigger compliance flags such as missing authentication, unencrypted data handling, or absence of audit trails.
For Android applications, installed apps are scanned by extracting and decompiling the Android program files, and reviewing the code structure for insecure practices such as hardcoded credentials or improper API calls. In both cases, the analysis method is performed without running any of the source code, allowing for the inspection of how HIPAA safeguards are configured.
The analysis engine then identifies implementation patterns by comparing the scanned code against a predefined rule library mapped to HIPAAChecker's twelve technical safeguards. Each violation is flagged within the report under its corresponding safeguard category, along with a short description of its compliance implication [4].
Next, the flagged issue is assigned a numerical severity score using the Common Vulnerability Scoring System (CVSS), which is integrated into HIPAAChecker's reporting model. CVSS is used to rate how critical each issue is, based on how easy it is to exploit, and the compromise potential of confidentiality, integrity, or availability. Issues with higher risk severity, such as unencrypted ePHI transfer, or missing access control mechanisms, are assigned higher scores to help teams prioritize remediation based on risk.

HIPAAChecker groups all detected vulnerabilities into four primary vulnerability categories (VCs), which are defined below.

Vulnerability Category	Description
VC1: Inadequate Data Security	Covers weaknesses in data protection, encryption, and cryptographic implementation
VC2: Insufficient Authorization	Involves inadequate access controls, credential verification, and authorization tracking
VC3: Insecure Network Communication	Includes unencrypted data transmission and weak SSL/TLS configurations that expose ePHI during transit
VC4: Inconsistent Audit Trail	Pertains to incomplete or missing logs of ePHI access, modifications, or other user-related events

Table 2: Vulnerability Categories and Descriptions

5. Results

5.1 Common Safeguard Violations

These findings were identified through repeated scans, and reflect similar weaknesses reported in previous evaluations. Access control, data encryption, and audit logging were frequently observed issues across tested applications.
To further verify these findings, the results in Table 4, indicate that all of the tested applications present a medium to high level

of compliance risk, as well as the overall risk scores for each sample application across the four vulnerability categories. Table 5 then provides a more in-depth risk breakdown, showing the risk percentage across four safeguard categories.

Safeguard Category	Implementation Flaw
Access Control	Lack of role enforcement, missing authorization middleware
Authentication	Use of weak or hardcoded credentials
Transmission Security	Incomplete HTTPS/TLS implementation
Data-at-rest Encryption	Storage of sensitive data as plain text
Audit Controls	Lack of audit logging
Session Management	Improper session handling

Table 3. Common safeguard violations in HIPAAChecker scans

App ID	App Name	Platform	Overall Risk Score	Risk Level
PYTHON001	Ehr-Django	Python-Django	25.7/100	Medium
LARAVEL001	Lhr-ehr-laravel	Laravel	41.4/100	Medium
DOTNET001	Hospital-Mgmt system	.NET	43.7/100	Medium
JAVA001	Healthcare Mgmt	Spring Boot	63.8/100	High

Table 4. HIPAA Risk Scores for Sample Applications

App Name	Insufficient Authorization	Inadequate Data Security	Insecure Network Comm.	Inconsistent Audit Trail
Ehr-Django	16.8%	15.3%	30.0%	**44.4%**
Lh-ehr-laravel	10.0%	0.0%	**75.0%**	**100%**
Hospital-Mgmt	22.6%	36.6%	13.3%	**100%**
Healthcare Mgmt.	10.4%	**66.8%**	**82.8%**	**100%**

Table 5. Risk Breakdown by Category

6. Discussion

6.1 Discussion of Results

The scan results indicate persistent compliance challenges observed in mHealth applications . As shown in Table 4, no application was fully compliant, and the risk scores confirm the tool's ability to effectively distinguish between the security postures of applications built on different technology stacks.

In Table 5, it was observed that an application's moderate overall risk score can actually overlook critical compliance failures in specific categories. For example, three of the four applications completely failed in the category of "Inconsistent Audit Trail," validating the lack of activity logging, which is a core requirement of HIPAA. Additionally,

the Healthcare Management app's high risk scores in "Inadequate Data Security" and "Insecure Network Communication" reveal critical vulnerabilities in compliance implementation that would be difficult to identify without a compliance auditor tool like HIPAAChecker.

Overall, the findings reveal the impact of small development teams, who make up a significant portion of the mHealth-related app market. For these developers, HIPAAChecker serves as a valuable and no-cost solution in implementing regulatory compliance, as it efficiently translates these requirements and provides actionable feedback. In this way, teams can quickly address vulnerability risk early in development.

Although HIPAAChecker is not yet a substitute for a formal audit, it does serve as an accessible tool that allows small-scale development teams to improve their security posture and better align with HIPAA's technical safeguards.

6.2 Limitations

During hands-on testing, several limitations of the HIPAAChecker platform were observed. Firstly, the tool occasionally failed to load or extract certain non-listed applications, such as an attempted scan of MyFitnessPal, which prevented it from being scanned. On occasion, the logged in session would reset, prompting the user to re-login again.

Further, the web portal and Android application versions do not synchronize reports, requiring manual cross-referencing of results.

One final limitation of HIPAAChecker is that it does not allow for generated reports to be exported. While these issues suggest areas for usability improvements, they do not detract from the accuracy and precision of the application.

7. Conclusions

7.1 Conclusion

To encapsulate this project, the findings from testing the HIPAAChecker platform validate its effectiveness as a tool for automating the detection of HIPAA technical safeguard violations in mHealth applications. Through repeated analysis of a curated set of applications, it was confirmed that significant shortcomings in compliance, particularly related to audit logging, access control, and data transmission are persistent in the mHealth ecosystem.

Further, the findings demonstrate that HIPAAChecker can quickly and accurately identify these specific flaws in implementation, and quantify their severity through the use of the CVSS framework.

For the developers who often lack the resources for formal compliance audits, the platform is an accessible tool that lowers the barrier to entry for security validation and provides actionable feedback to build more secure and compliant applications from the start.

7.2 Future Work

The work completed in this project points to several paths for future development. The immediate priority for the platform is to move beyond a research prototype and towards a more commercialized product, which will require secured investment to expand its features and promote its marketability. Additionally, a key part of expansion efforts currently includes completing the iOS version of HIPAAChecker, which is currently in development and will significantly expand the platform's reach potential.

Additional future work could focus on incorporating dynamic analysis to complement the existing static analysis engine.

In this way, it is not only recognized as industry standard best practice, it also provides a more comprehensive security assessment by covering runtime vulnerabilities that static analysis alone cannot identify (OWASP, 2022).

8. References

[1] M. R. Mia, H. Shahriar, M. Valero, *et al.*, "A comparative study on HIPAA technical safeguards assessment of Android mHealth applications," *Smart Health*, vol. 26, p. 100349, 2022. [Online]. Available: https://doi.org/10.1016/j.smhl.2022.100349

[2] S. Akter, M. A. Barek, M. M. Rahman, A. K. I. Riad, M. A. Rahman, M. R. Mia, H. Shahriar, W. Chu, and S. I. Ahamed, "HIPAA Technical Compliance Evaluation of Laravel-based mHealth Apps," in *Proc. 2024 IEEE Int. Conf. on Digital Health (ICDH)*, 2024. [Online]. Available: https://doi.org/10.1109/ICDH62654.2024.00020

[3] A. K. I. Riad, M. A. Barek, M. M. Rahman, M. S. Akter, T. Islam, M. A. Rahman, M. R. Mia, H. Shahriar, F. Wu, and S. I. Ahamed, "Enhancing HIPAA compliance in AI-driven mHealth devices: Security and privacy," in *Proc. 2024 IEEE 48th Annu. Comput., Softw. & Appl. Conf. (COMPSAC)*, 2024. [Online]. Available: https://doi.org/10.1109/COMPSAC61105.2024.00390

[4] B. Saha, S. Tahora, A. Barek, and H. Shahriar, "HIPAAChecker: The comprehensive solution for HIPAA compliance in Android mHealth apps," in *Proc. 2023 IEEE 47th Annu. Comput., Softw. & Appl. Conf. (COMPSAC)*, 2023. [Online]. Available: https://doi.org/10.1109/COMPSAC57700.2023.00282

[5] ORCHA Health, "Market Facts & Figures," 2025. [Online]. Available:https://orchahealth.com/market-facts-and-figures

[6] IBM Security, "Guardium Data Compliance." [Online]. Available: https://www.ibm.com/products/guardium/data-compliance

[7] U.S. Government Publishing Office, "45 CFR § 164.312 - Technical safeguards," *Electronic Code of Federal Regulations*, [Online]. Available:https://www.law.cornell.edu/cfr/text/45/164.312

[8] OWASP Foundation, "Source Code Analysis Tools," OWASP.org, 2022. [Online]. Available: https://owasp.org/www-community/Source_Code_Analysis_Tools